Dear participant,

We are looking forward to welcoming you to the **Claude Code Mastery Training** training. To make the most of our time together, we ask you to complete a few preparation steps before the session.

What to prepare

1. An editor or IDE

Use the editor or IDE you like best, as long as it can open and browse Kotlin code. During the training the editor is only used to read code and review changes. All AI work happens in Claude Code in the terminal.

If you have no preference, Visual Studio Code is a safe choice. The instructions below use VSCode where a step depends on the editor.

2. Kotlin

Make sure the **Kotlin compiler** (`kotlinc` 1.9+) and a **JDK** (Java 21+) are installed on your machine:

- Kotlin: https://kotlinlang.org/docs/command-line.html
- JDK 21+: https://adoptium.net/ (Temurin builds work well)

Verify both work by running:

```
kotlinc -version
java -version
```

Your IDE also needs to understand Kotlin. IntelliJ IDEA supports it out of the box, including the free Community Edition. In VSCode, install a **Kotlin extension**:

- Open the Extensions panel (`Ctrl+Shift+X` / `Cmd+Shift+X`)
- Search for **"Kotlin"** (the *Kotlin Language* extension by mathiasfrohlich is a good lightweight option)
- Click **Install**

3. Claude Code subscription and plugins

We will be using **Claude Code** as our AI coding assistant during the training. Please set up your account and tools before the session:

- Sign up or log in at claude.ai
- Make sure you have an active **Claude Pro** subscription (required for Claude Code access)
- Install the **Claude Code CLI** by following the instructions at claude.ai/code
- Verify it works by running `claude` in your terminal
- Optional: Claude Code has editor integrations for VSCode and for JetBrains IDEs. They are convenient but not required — the terminal is enough for every exercise. In VSCode, open the Extensions panel (`Ctrl+Shift+X` / `Cmd+Shift+X`), search for **"Claude Code"** by Anthropic, and click **Install**.

4. Git

Make sure **Git** is installed on your machine: https://git-scm.com/downloads

Verify it works by running:

```
git --version
```

5. Course materials on GitHub

We will be working with the **BookStore API** starter project during the training. The repository has one branch per language. Clone the Kotlin one into a folder named `bookstore` , before the session:

```
git clone -b kt https://github.com/xebia/ai-assisted-se-claude-exercise bookstore
```

Then enter the folder and build it to verify everything works (the first build downloads the `sqlite-jdbc` JAR):

```
cd bookstore
./gradlew run
# or: ./mvnw package -DskipTests && java -jar target/bookstore-1.0-SNAPSHOT.jar
```

6. uv, Spec Kit and Node (for Exercises 4 to 7)

In Exercise 6 we use Spec Kit, GitHub's spec-driven development toolkit, to specify a frontend for the BookStore API.

Node.js 20+ — https://nodejs.org/ (the frontend's dev server runs on it)

uv — Exercise 4 starts a small Python MCP server with `uv`, Exercise 5 runs its gate and brief scripts with `uv`, and Spec Kit is a Python tool installed via `uv`:

```
curl -LsSf https://astral.sh/uv/install.sh | sh     # macOS / Linux
# Windows (PowerShell):
# powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/0.9.9/install.ps1 | iex"
```

Then install Spec Kit and set up the frontend project:

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git

cd bookstore/web
npm install
specify init --here --force --non-interactive --integration claude
git checkout -- .specify/memory/constitution.md      # must run AFTER init
specify check
```

Exercise 7 uses **agent teams**, an experimental Claude Code feature that is off by default. The file `web/.claude/settings.json` is committed and turns it on for that project only. It needs Claude Code **2.1.178 or newer**. Check both:

```
claude --version
git status --short .claude/settings.json      # must print nothing
```

If `specify init` changed or removed the file, restore it:

```
git checkout -- .claude/settings.json
```

> [!IMPORTANT] The `git checkout` must come **after** `specify init`. The constitution is committed at `.specify/memory/constitution.md`, and `specify init --force` overwrites it with its own template. Restoring it from git puts the real one back — and `git status` will tell you if you forget.

Please do this **before the training day**. `specify init` downloads a template bundle from GitHub, and thirty laptops doing that at once on conference wifi is not a good start to a session.

Verify: `specify check` reports no problems, and `npm run dev` starts a server on http://localhost:5173.

Quick checklist

- Your editor or IDE is installed and can open Kotlin files
- Kotlin compiler is installed (`kotlinc -version` works in your terminal)
- JDK 21+ is installed (`java -version` shows 21 or higher)
- You have an active Claude Pro subscription ($20/month)
- Claude Code CLI is installed and working (`claude --version` works in your terminal)
- Optional: the Claude Code editor integration is installed
- Git is installed (`git --version` works in your terminal)
- You have cloned the `kt` branch of the exercise repository into a folder named `bookstore`, and the project runs and `./gradlew run` (or `./mvnw package -DskipTests`) finishes without errors
- Node.js 20+ is installed (`node --version` works in your terminal)
- `uv` is installed (`uv --version` works in your terminal)
- Spec Kit is installed (`specify check` runs without problems)
- `bookstore/web` is set up (`npm install` finished, `.specify/` exists, `.specify/memory/constitution.md` starts with "# BookStore Web Constitution" and `git status` shows it unmodified)
- Claude Code is 2.1.178 or newer, and `web/.claude/settings.json` is present and unmodified (`git status` shows nothing for it)

Questions?

If you run into any issues during setup, feel free to reach out to us before the training day. We want everyone to hit the ground running from the very first exercise.

See you soon!

The Training Team